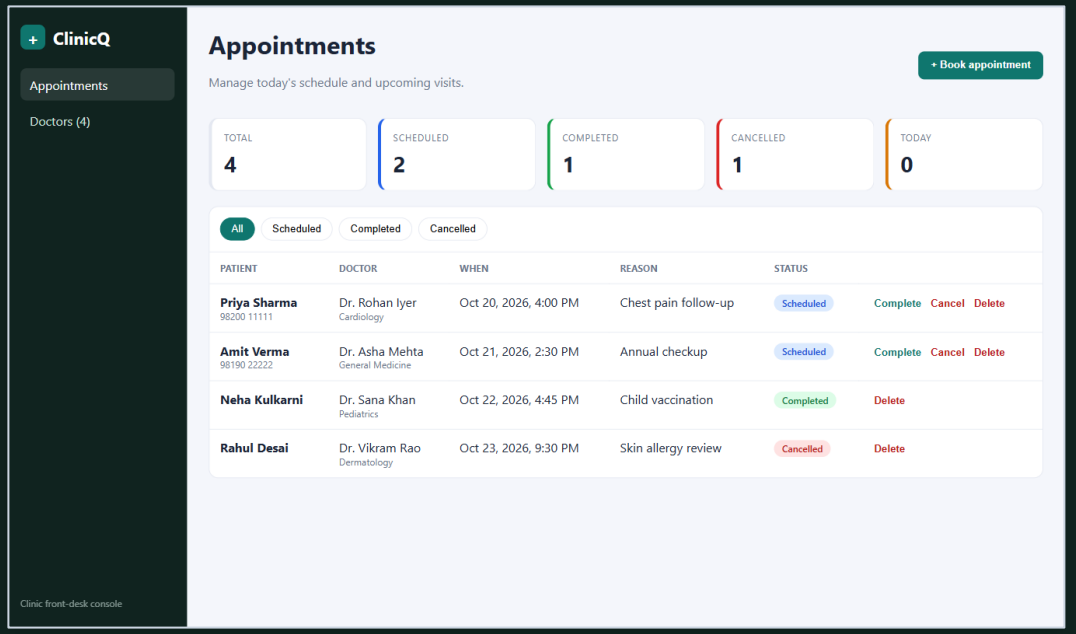
ClinicQ

Clinic appointment booking, taken from a laptop to a monitored Kubernetes cluster on AWS



DevOps Final Capstone

Aryan Bhendarkar
github.com/Aryan-Bhendarkar/ClinicQ

What I built

- **The problem:** front desks double-book doctors when schedules live on paper or in chat.
- **The app:** a booking console with stats, filters, status updates, and a 409 conflict guard per doctor and slot.
- **The stack:** React + Vite, FastAPI, PostgreSQL, SQLAlchemy and Alembic.
- **The platform:** Docker, GitHub Actions, Trivy, GHCR, Terraform, EKS, Helm, Prometheus and Grafana.

12

REST endpoints

15

automated tests

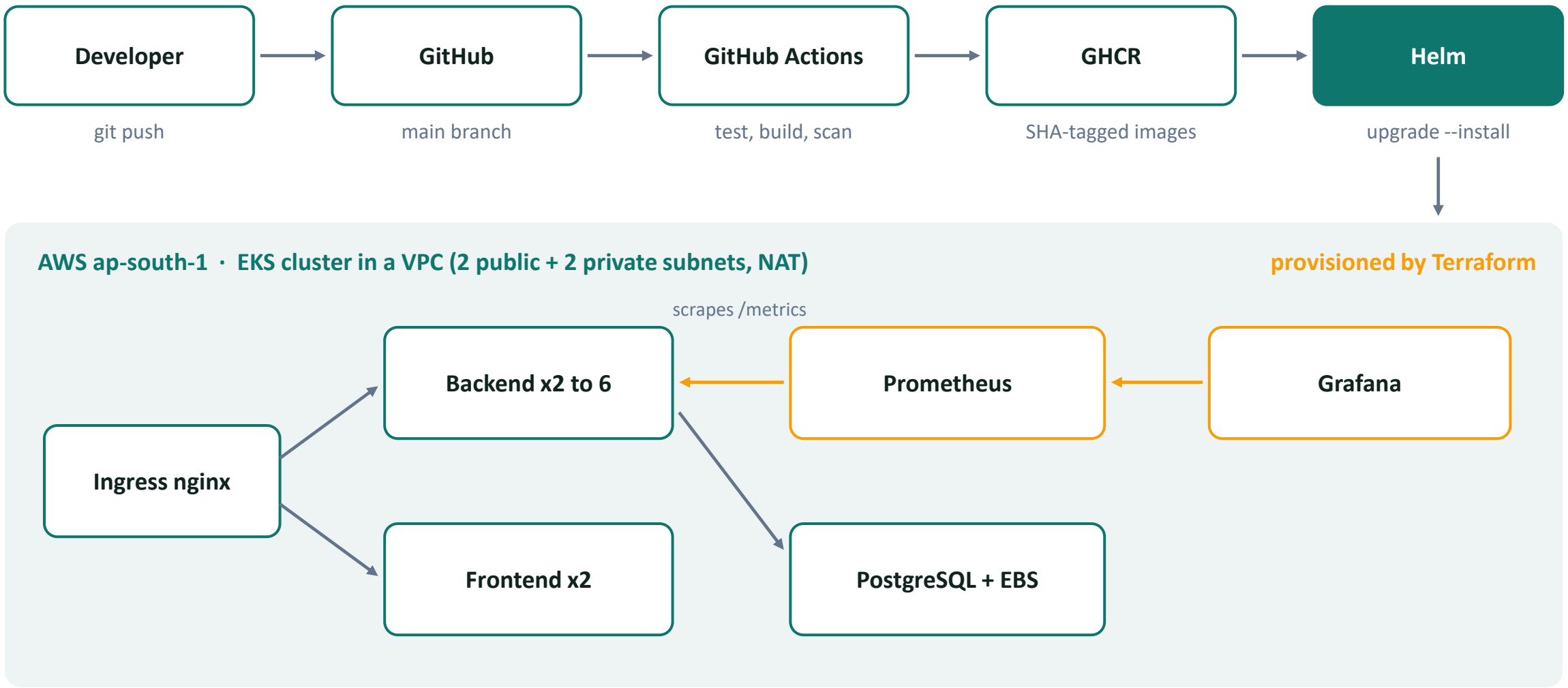
2

Alembic migrations

20+

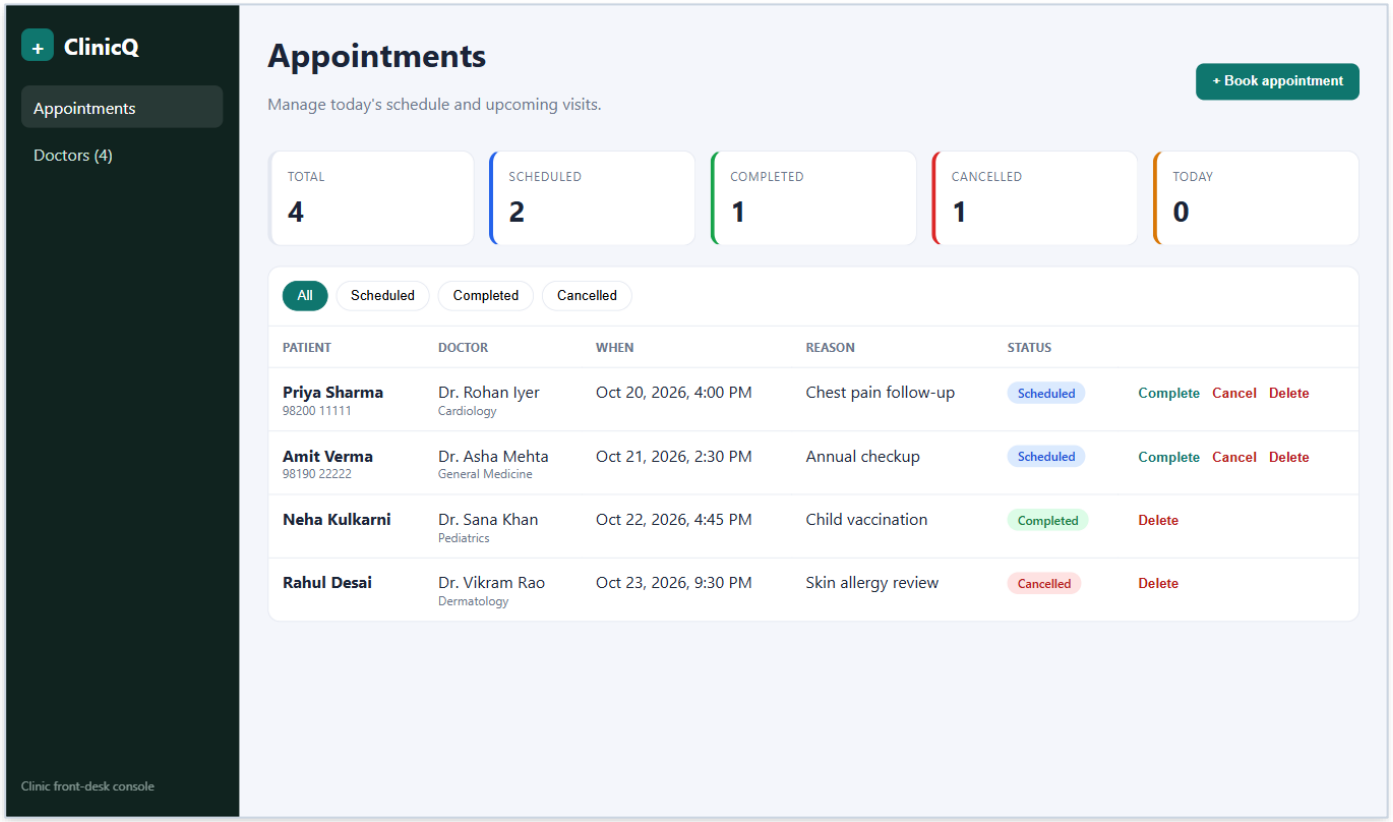
descriptive commits

Architecture



The application

M1 · App · 10 pts

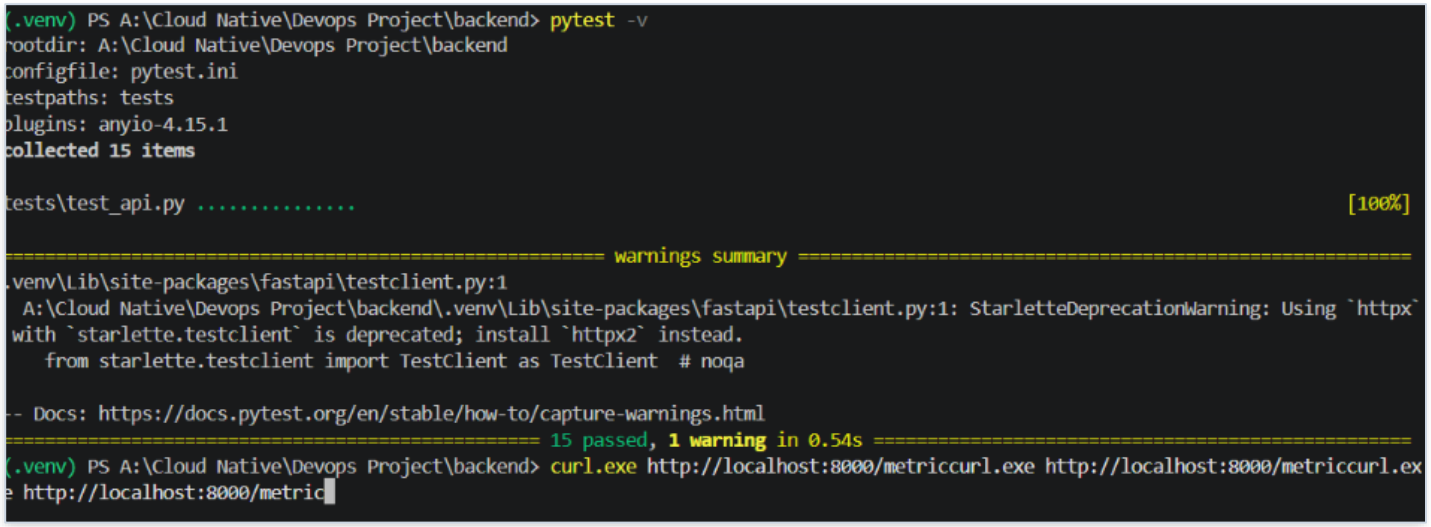


Docker Compose stack at localhost:3000

- **React + Vite UI:** KPI cards, status filters, booking modal, responsive layout.
- **FastAPI:** 12 endpoints; Swagger at /docs; /health, /ready, /metrics.
- **Alembic:** schema migration plus a seed migration for doctors.
- **Conflict guard:** double-booking a doctor's slot returns 409.

Automated tests

M2 · Testing · 10 pts



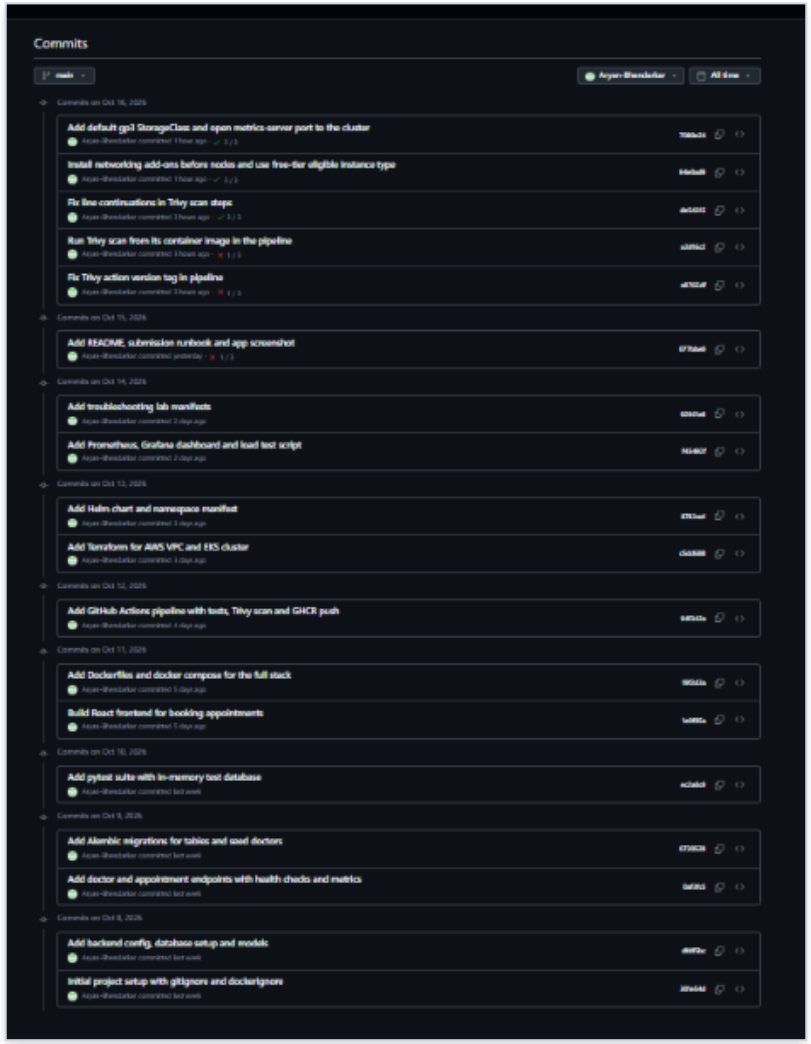
pytest -v: 15 passed

- **15 tests** covering health, readiness, metrics, doctors and every appointment endpoint.
- **Isolated:** in-memory SQLite through a FastAPI dependency override, never the real database.
- **Configured:** pytest.ini and conftest.py.

First quality gate: failing tests stop the pipeline, so no broken image is ever built or pushed.

Git and GitHub

M3 · Git · 5 pts



20+

commits, Oct 8 to Oct 16

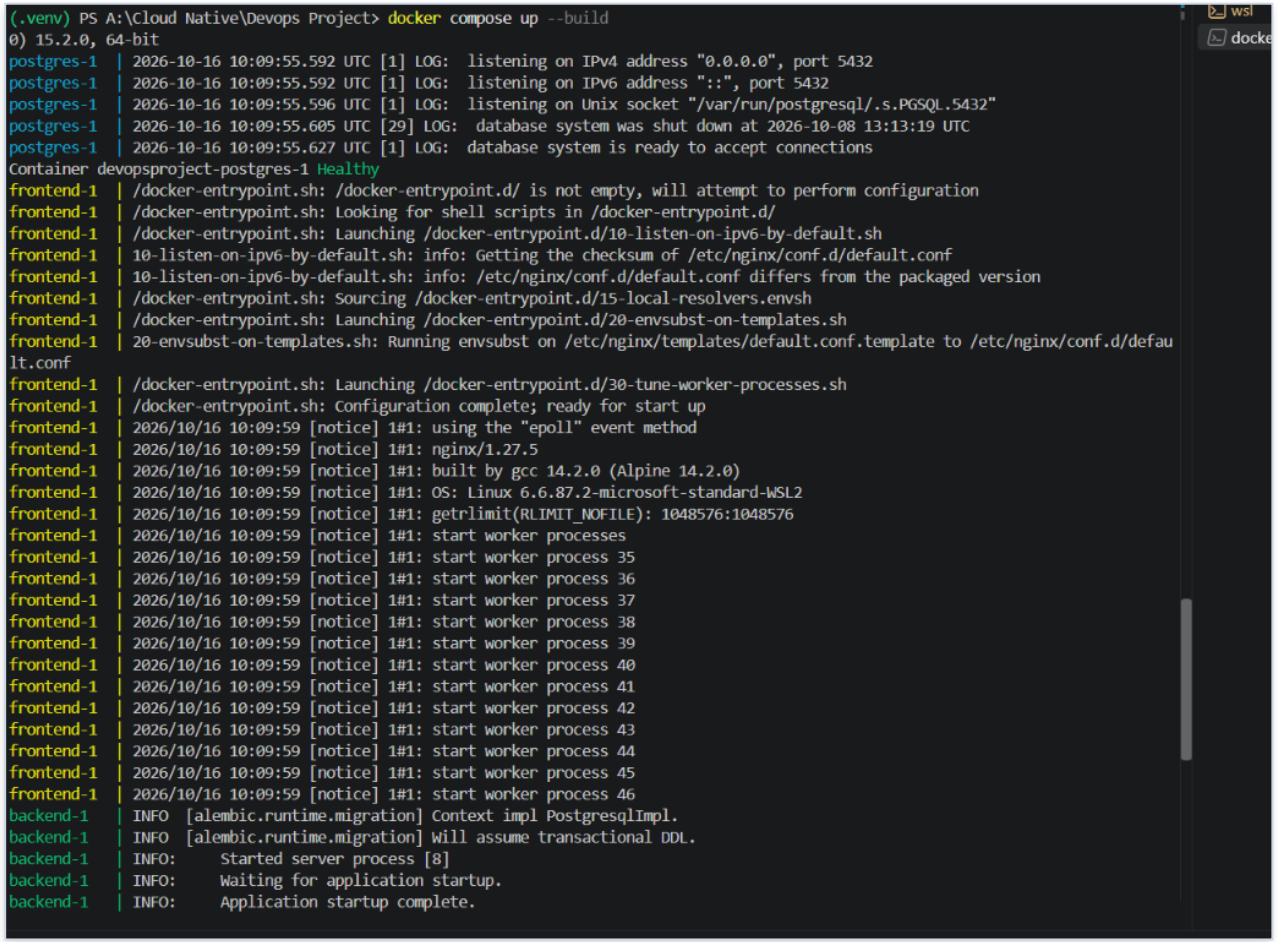
0

secrets in the history

- **Messages:** short and descriptive, one per feature or fix.
- **.gitignore** covers .env, __pycache__, node_modules, .venv, Terraform state and plans.
- **Public repo:** github.com/Aryan-Bhendarkar/ClinicQ

Docker and Compose

M4 · Docker · 10 pts



docker compose up --build

- **Multi-stage frontend:** Node builds, unprivileged Nginx serves.
- **Non-root:** backend runs as uid 10001, frontend as uid 101.
- **One command:** postgres, backend and frontend; the backend waits for a healthy database.
- **Health checks** on Postgres and the backend.

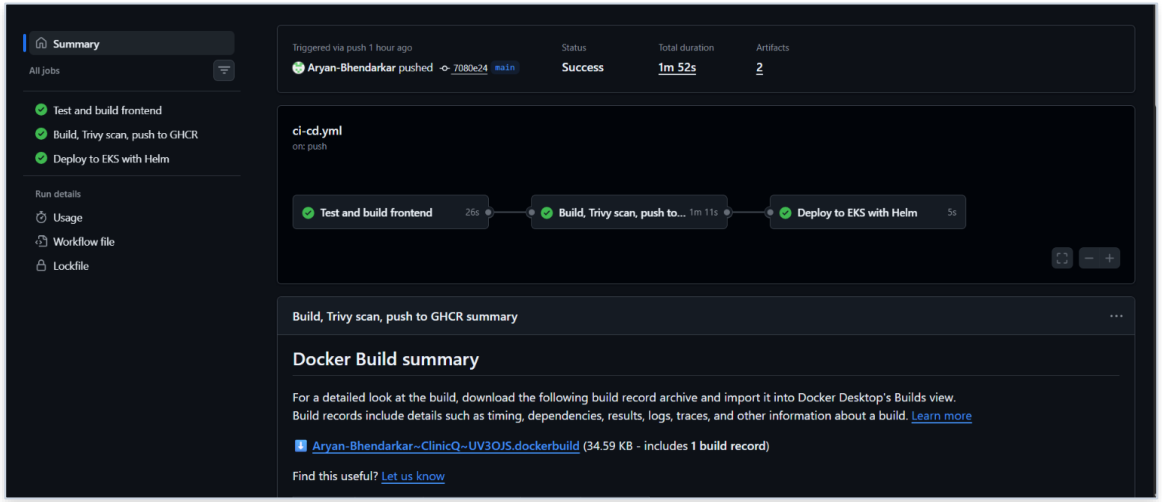
10001
backend uid

101
frontend uid

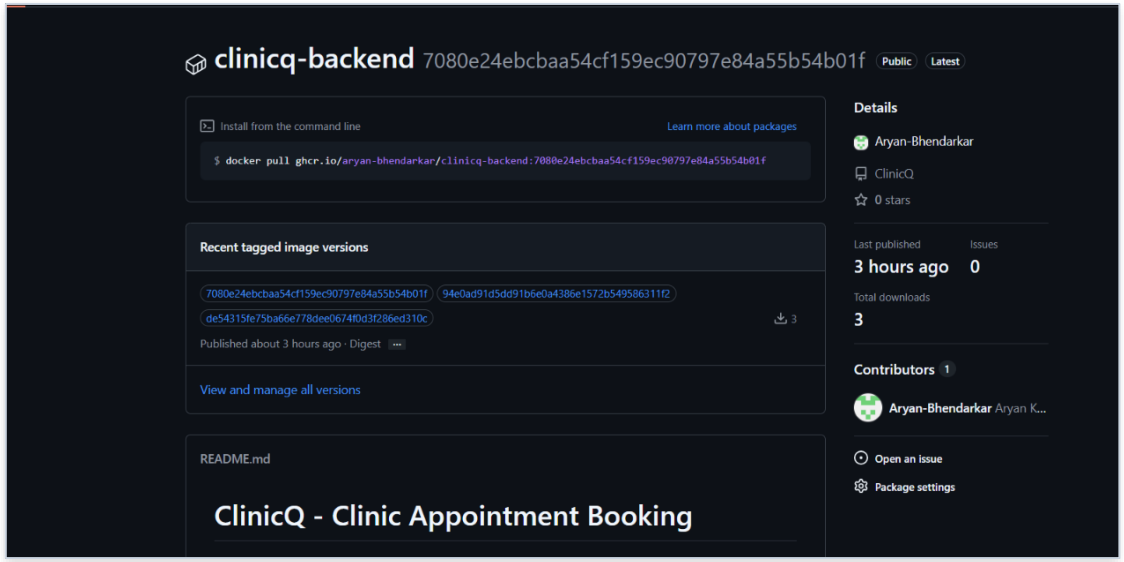
CI/CD pipeline

M5 · CI/CD · 15 pts

pytest → Frontend build → Docker build x2 → Trivy scan x2 → Push to GHCR



Green run: all three jobs



GHCR images tagged with the commit SHA

Triggers on every push to main. Nothing is pushed unless tests and both scans pass; tags are commit SHAs, never latest.

Security scanning with Trivy

M6 · Security · 5 pts

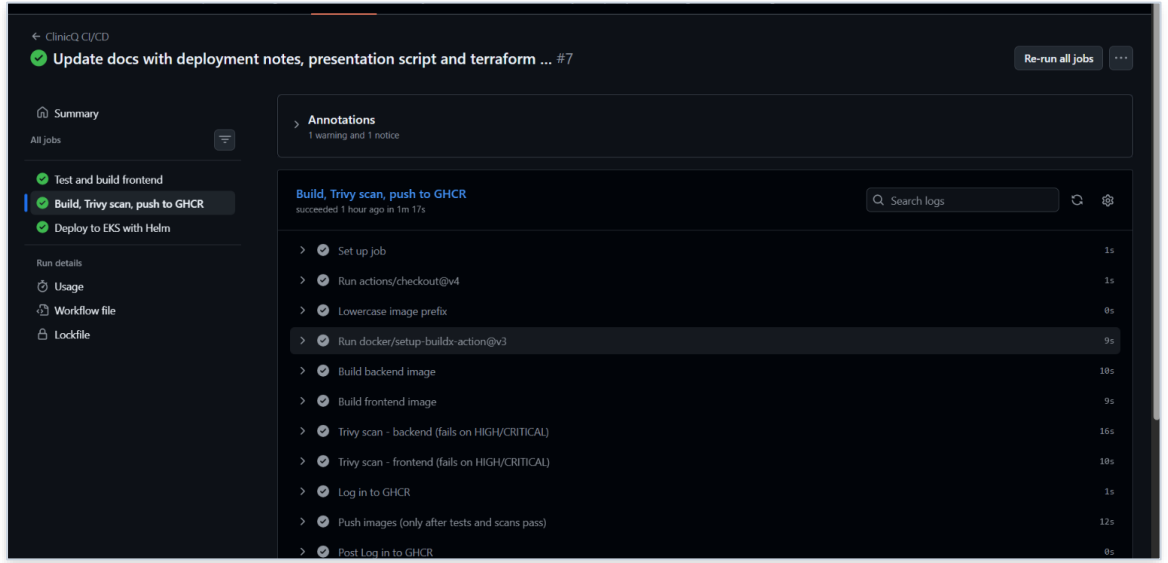
3 → 0

HIGH CVEs, before and after

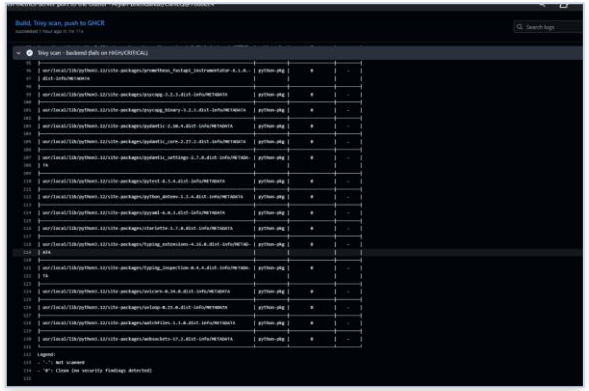
2 / 2

images scanned in CI

- **Found:** 3 HIGH CVEs in Starlette (via an old FastAPI pin); zlib and nghttp2 in the Alpine image.
- **Fixed:** upgraded FastAPI, Starlette and the instrumentator; apk upgrade in the runtime image.
- **Gate:** --exit-code 1 fails the build on any HIGH or CRITICAL finding.



Both scans run before the push step

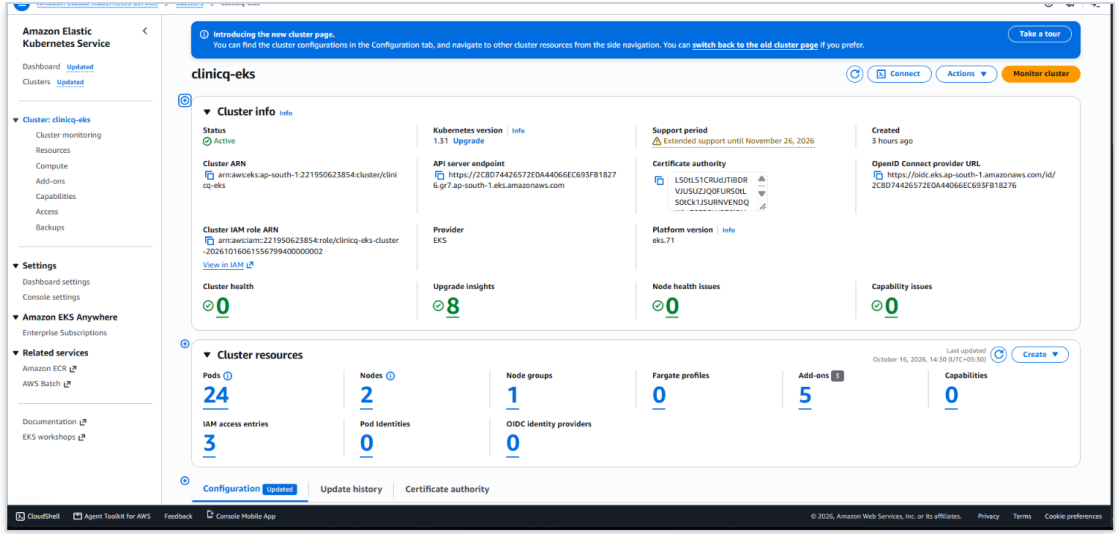


Backend: zero findings

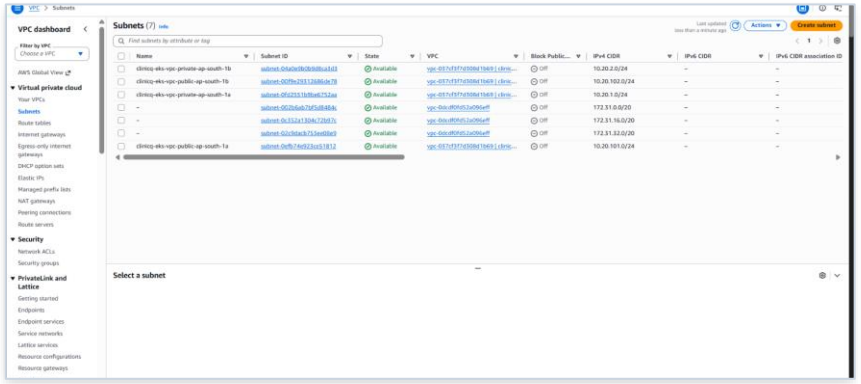
Terraform on AWS

M7 · Terraform · 15 pts

- **VPC:** 2 public + 2 private subnets, internet gateway, NAT.
- **EKS 1.31:** managed node group, 2 x m7i-flex.large.
- **Add-ons:** vpc-cni, kube-proxy, CoreDNS, EBS CSI, metrics-server.
- **Variables:** terraform.tfvars.example, no credentials in Git.



EKS cluster clinicq-eks: Active, ap-south-1



VPC subnets: two public, two private

Plan, apply, destroy

M7 · Terraform · 15 pts

```
        + "terraform-aws-modules" = "eks"
      }
    }

  # module.eks.module.eks_managed_node_group["main"].module.user_data.null_resource.validate_cluster_service_cidr will be created
  + resource "null_resource" "validate_cluster_service_cidr" {
      + id = (known after apply)
    }

Plan: 60 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + cluster_endpoint   = (known after apply)
  + cluster_name       = "clinicq-eks"
  + configure_kubectl  = "aws eks update-kubeconfig --region ap-south-1 --name clinicq-eks"
  + private_subnet_ids = [
      + (known after apply),
      + (known after apply),
    ]
  + public_subnet_ids  = [
      + (known after apply),
      + (known after apply),
    ]
  + vpc_id             = (known after apply)

Warning: Argument is deprecated

  with module.eks.aws_iam_role.this[0],
  on .terraform\modules\eks\main.tf line 402, in resource "aws_iam_role" "this":
 402: resource "aws_iam_role" "this" {

inline_policy is deprecated. Use the aws_iam_role_policy resource instead. If Terraform should exclusively manage all inline policy associations
(the current behavior of this argument), use the aws_iam_role_policies_exclusive resource as well.
```

terraform plan: 60 to add, 0 errors

```
(.venv) PS A:\Cloud Native\Devops Project\terraform> terraform apply
module.eks.aws_eks_addon.this["coredns"]: Creating...
module.eks.aws_eks_addon.this["aws-ebs-csi-driver"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["metrics-server"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Creation complete after 14s [id=clinicq-eks:coredns]
module.eks.aws_eks_addon.this["aws-ebs-csi-driver"]: Still creating... [00m20s elapsed]
module.eks.aws_eks_addon.this["metrics-server"]: Still creating... [00m20s elapsed]
module.eks.aws_eks_addon.this["aws-ebs-csi-driver"]: Creation complete after 24s [id=clinicq-eks:aws-ebs-csi-driver]
module.eks.aws_eks_addon.this["metrics-server"]: Creation complete after 25s [id=clinicq-eks:metrics-server]
module.eks.module.eks_managed_node_group["main"].aws_eks_node_group.this[0] (deposed object 6f37ea4c): Destroying... [id=clinicq-eks
:main-20261016063257349700000004]
module.eks.module.eks_managed_node_group["main"].aws_eks_node_group.this[0]: Destruction complete after 8s

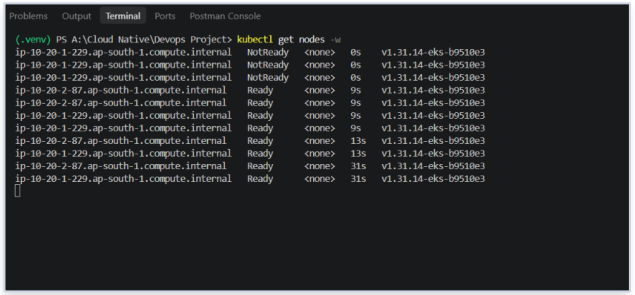
Apply complete! Resources: 4 added, 0 changed, 4 destroyed.

Outputs:

cluster_endpoint = "https://2C8D74426572E0A44066EC693FB18276.gr7.ap-south-1.eks.amazonaws.com"
cluster_name = "clinicq-eks"
configure_kubectl = "aws eks update-kubeconfig --region ap-south-1 --name clinicq-eks"
private_subnet_ids = [
  "subnet-0fd2551b9be6752aa",
  "subnet-04a0e9b0b9d8ca3d3",
]
public_subnet_ids = [
  "subnet-0efb74e923ce51812",
  "subnet-00f9e29312686de78",
]
vpc_id = "vpc-037cf3f7d308d1b69"
```

terraform apply: complete, with outputs

- **Plan first,** then apply the saved plan.
- **Destroy** removes everything, so there is no runaway AWS cost.



kubectl get nodes: both nodes Ready

Kubernetes and Helm

M8 · K8s + Helm · 15 pts

- **Helm chart:** Deployments, ClusterIP Services, Ingress, HPA, ServiceMonitor.
- **PostgreSQL:** StatefulSet on an EBS gp3 volume.
- **Migrations:** a Helm hook Job, so replicas never race.
- **Replicas:** 2 backend and 2 frontend.
- **Namespace** and StorageClass manifests in k8s/.

```
ip-10-20-2-87.ap-south-1.compute.internal    Ready      <none>   46m    v1.31.14-eks-b9510e3
(.venv) PS A:\Cloud Native\Devops Project> kubectl get pods -n clinicq
NAME                                 READY   STATUS    RESTARTS   AGE
clinicq-backend-6dd8569fb6-97l7c     1/1     Running   0          45m
clinicq-backend-6dd8569fb6-cd7wc     1/1     Running   0          60m
clinicq-frontend-6cbcc866f8-svr2p    1/1     Running   0          60m
clinicq-frontend-6cbcc866f8-t4866    1/1     Running   0          60m
clinicq-postgres-0                   1/1     Running   0          60m
(.venv) PS A:\Cloud Native\Devops Project>
```

kubectl get pods -n clinicq

```
(.venv) PS A:\Cloud Native\Devops Project> kubectl get svc -n clinicq
NAME               TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
clinicq-backend    ClusterIP   172.20.63.205    <none>        8000/TCP   60m
clinicq-frontend   ClusterIP   172.20.94.186    <none>        80/TCP     60m
clinicq-postgres   ClusterIP   172.20.182.165   <none>        5432/TCP   60m
(.venv) PS A:\Cloud Native\Devops Project>
```

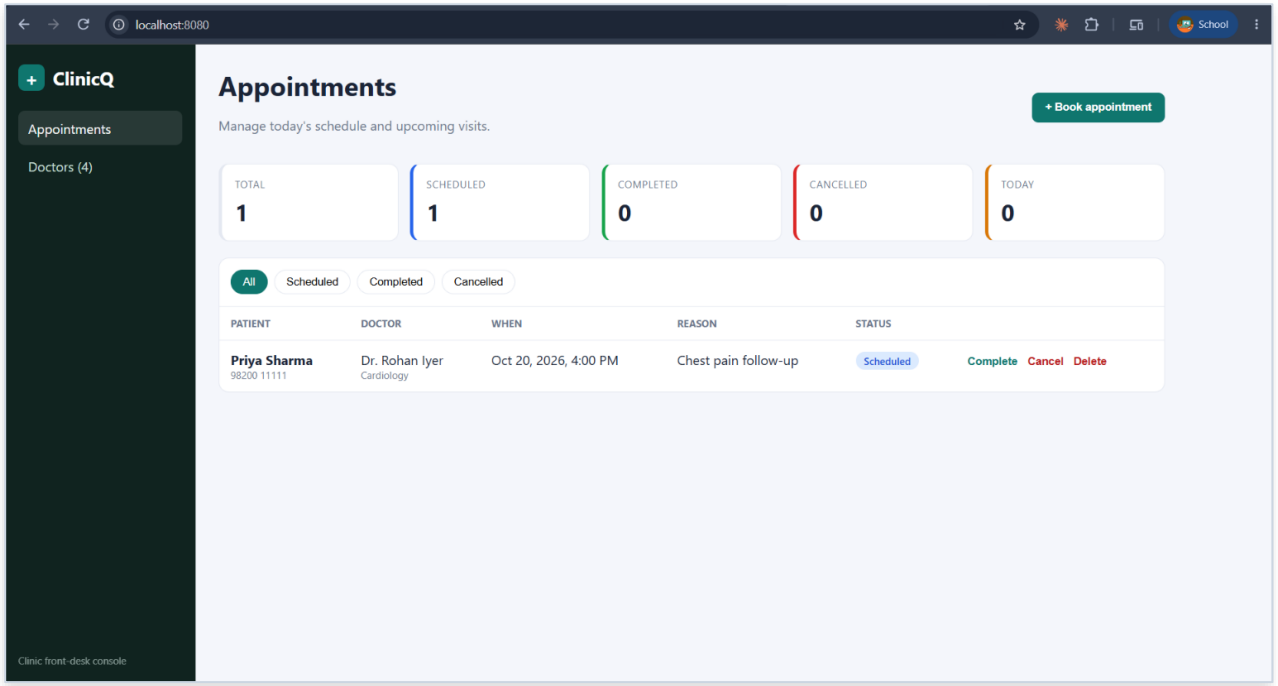
kubectl get svc -n clinicq

```
(.venv) PS A:\Cloud Native\Devops Project> helm list -n clinicq
NAME    NAMESPACE       REVISION        UPDATED                                 STATUS          CHART           APP VERSION
clinicq clinicq         2               2026-10-16 13:02:30.7339138 +0530 IST   deployed        clinicq-1.0.0   1.0.0
(.venv) PS A:\Cloud Native\Devops Project>
```

helm list -n clinicq

Ingress and autoscaling

M8 · K8s + Helm · 15 pts



App served through the Ingress

```
(.venv) PS A:\Cloud Native\Devops Project> kubectl get hpa -n clinicq
NAME              REFERENCE                    TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
clinicq-backend   Deployment/clinicq-backend   cpu: 2%/50%   2         6         2          61m
(.venv) PS A:\Cloud Native\Devops Project>
```

kubectl get hpa: min 2, max 6

2 → 6

backend pods under load

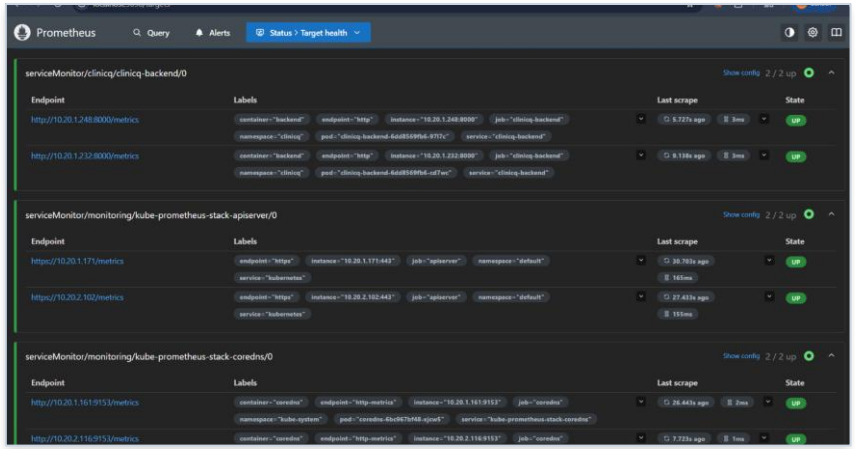
50%

CPU target

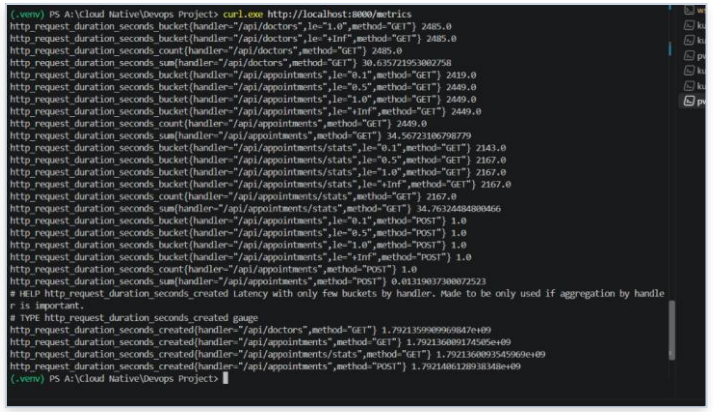
- **Ingress:** /api goes to the backend, everything else to the frontend.
- **HPA:** scaled the backend from 2 to 6 pods during a load test.
- **Note:** this AWS account cannot create load balancers, so I reach the Ingress with a port-forward.

Monitoring

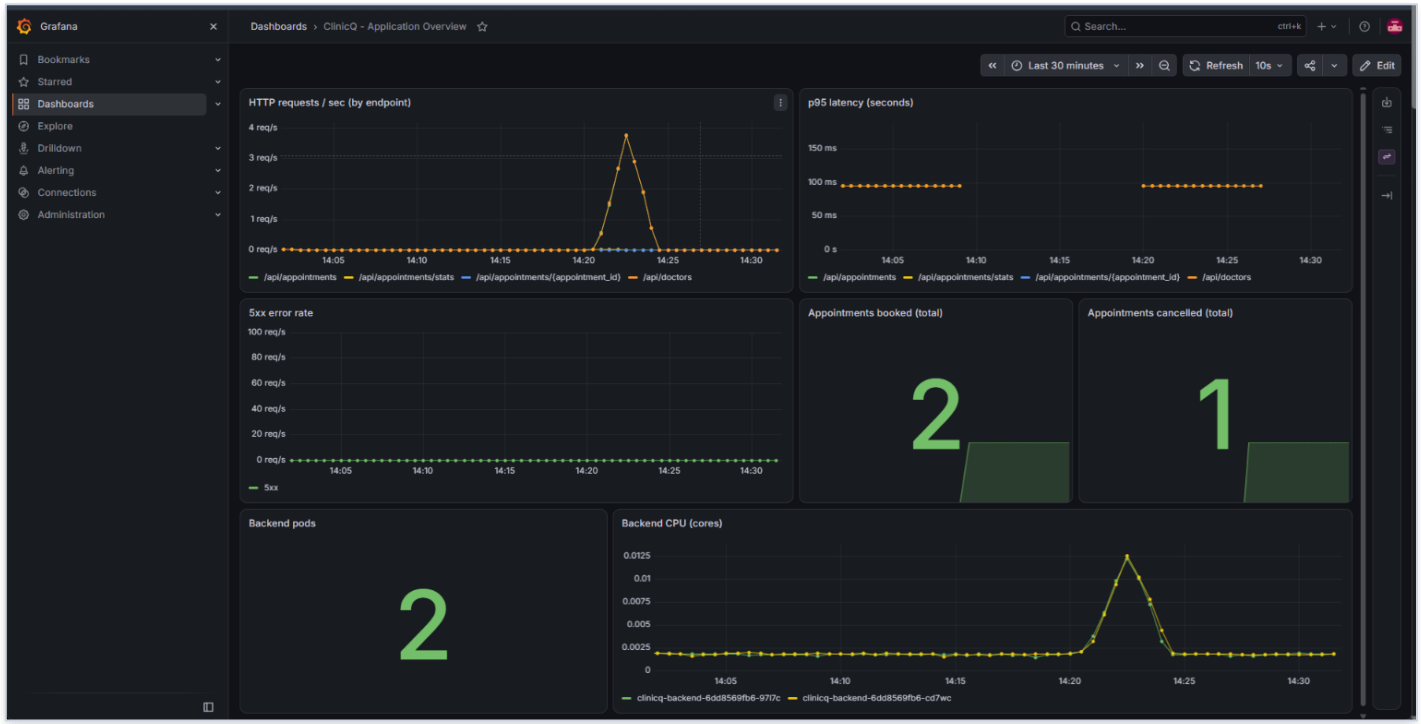
M9 · Observability · 10 pts



Prometheus: clinicq-backend 2/2 UP



/metrics in Prometheus format



Grafana: request rate, p95 latency, 5xx, bookings, pods, CPU

A ServiceMonitor tells Prometheus to scrape both backend pods. A custom counter tracks bookings and cancellations.

Problems I solved

Symptom	Root cause	Fix
Node group stuck CREATING	Free Tier plan rejects t3.medium	Use m7i-flex.large
Nodes never Ready	Networking add-ons created after the nodes	before_compute for vpc-cni and kube-proxy
Postgres volume Pending	No default StorageClass for the CSI driver	Default gp3 StorageClass
HPA showed <unknown>	Port 10251 to metrics-server was blocked	Security group rule in Terraform
Ingress had no address	Account cannot create load balancers	Port-forward to the Ingress controller
Trivy and Terraform errors	Bad action tag; expiring credentials	Trivy container; auto-refreshing profile

Troubleshooting lab

1 Bad image tag

Symptom
ImagePullBackOff

Diagnose
```
kubectl describe pod

kubectl get events
```

Fix
Correct the image tag

2 Service has no endpoints

Symptom
Label mismatch

Diagnose
```
kubectl get endpoints

kubectl get pods --show-labels
```

Fix
Patch the Service selector

3 Pod never Ready

Symptom
Wrong readiness path

Diagnose
```
kubectl describe pod

look at probe events
```

Fix
Fix the probe path

Limits and next steps

Known limits

- The AWS account cannot create load balancers, so the Ingress is reached by port-forward.
- The pipeline's deploy job needs a kubeconfig secret that I did not set; I deployed with Helm from my laptop using the pipeline's images.
- PostgreSQL runs inside the cluster for a self-contained capstone.

What I would do next

- Amazon RDS for backups and Multi-AZ failover.
- A real load balancer with DNS and TLS.
- Secrets in AWS Secrets Manager.
- OIDC-based credentials so the pipeline can deploy without stored secrets.
- Alert rules for error rate and latency.

Demo and questions

Commit, pipeline, image, Helm deploy, Ingress, Grafana, then a live break and fix.

✓ M1 App	✓ M2 Tests	✓ M3 Git	✓ M4 Docker	✓ M5 CI/CD
✓ M6 Trivy	✓ M7 Terraform	✓ M8 K8s + Helm	✓ M9 Monitoring	✓ M10 Docs + demo

github.com/Aryan-Bhendarkar/ClinicQ